

Problem Description:

Good news! Suresh get to go to America on a class trip! Bad news, he don't know how to use the Dollar which is the name of the American cash system. America uses coins for cash a lot more than the Kuwait does. Dollar comes in coins for values of: 1, 2, 10, 50, 100, & 500 To practice your Dollar skills, suresh have selected random items from Amazon.co.us and put them into a list along with their prices in Dollar. Suresh now want to create a program to check suresh Dollar math.

Suresh goal is to maximize your buying power to buy AS MANY items as you can with your available Dollar.

Input Format:

File listing 2 to 6 items in the format of:

ITEM DDDDD

ITEM = the name of the item you want to buy

DDDDD = the price of the item (in Dollar)

Output Format:

Print the output in a separate lines contains, List the items suresh can afford to buy. Each item on its own line. Suresh goal is to buy as many items as possible. If suresh can only afford the one expensive item, or 2 less expensive items on a list, but not all three, then list the less expensive items as affordable. If suresh cannot afford anything in the list, output "I need more Dollar!" after the items. The final line you output should be the remaining Dollar he will have left over after make purchases.

Logical Test Cases

Test Case 1

INPUT (STDIN)

6000 3
Phone-case 1486
Candybar 863
Sunglasses 5529

EXPECTED OUTPUT

I can afford Phone-case
I can afford Candybar
I can't afford Sunglasses
3651

Test Case 2

INPUT (STDIN)

2100 3
Camera 69555
TV 76439
iPhone 90000

EXPECTED OUTPUT

I can't afford Camera
I can't afford TV
I can't afford iPhone
I need more Dollar!

Mandatory Test Cases

Test Case 1

KEYWORD

char name[MAX] [LEN];

Test Case 2

KEYWORD

int price[MAX]

Test Case 3

KEYWORD

afford [MAX]

Test Case 4

KEYWORD

for(i=0;i<items;i++)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

11

Test Case 2

TOKEN COUNT

250

Test Case 3

NLOC

33

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 100
5 #define LEN 100
6
7 int main() {
8     int total_dollar, items;
9     if (scanf("%d %d", &total_dollar, &items) != 2) return 0;
10
11     char name[MAX][LEN];
12     int price[MAX];
13     int afford[MAX] = {0};
14     int orig_order[MAX];
15     int sorted_indices[MAX];
16
17     int i, j;
18     for (i = 0; i < items; i++) {
19         scanf("%s %d", name[i], &price[i]);
20         orig_order[i] = i;
21         sorted_indices[i] = i;
22     }
23
24     for (i = 0; i < items; i++) {
25         for (j = i + 1; j < items; j++) {
```

```
26         if (price[sorted_indices[i]] > price[sorted_indices[j]]) {
27             int temp = sorted_indices[i];
28             sorted_indices[i] = sorted_indices[j];
29             sorted_indices[j] = temp;
30         }
```

main | CH.SC.U4CSE25225 | Array6.c

Ln 60, Col 1 Spaces: 4 UTF-8 CRLF C

```
32     }
33
34     int remaining = total_dollar;
35     int bought = 0;
36     for (i = 0; i < items; i++) {
37         int idx = sorted_indices[i];
```

```
38     if (price[idx] <= remaining) {
39         afford[idx] = 1;
40         remaining -= price[idx];
41         bought++;
42     }
43 }
44
45 for (i = 0; i < items; i++) {
46     if (afford[i]) {
47         printf("I can afford %s\n", name[i]);
48     } else {
49         printf("I can't afford %s\n", name[i]);
```

```
50     }
51 }
52
53 if (bought == 0) {
54     printf("I need more Dollar!\n");
55 }
56 printf("%d\n", remaining);
57
58 return 0;
59 }
60
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

6000 3

Phone-case 1486

Candybar 863

Sunglasses 5529

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array6.exe

I can afford Phone-case

I can afford Candybar

I can't afford Sunglasses

3651

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

2100 3

Camera 69555

TV 76439

iPhone 90000

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array6.exe

I can't afford Camera

I can't afford TV

I can't afford iPhone

I need more Dollar!

2100

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>